

Class 01 | 26 July 2026

we will start sharp at 09: 05 PM

Today's agenda

- Prepare yourself from today
- Understand how modern web applications work.
- Get an overview of the tools, technologies, and libraries we'll use.
- Analyze the requirements behind popular Twitter features.
- See how each Twitter feature maps to the concepts we'll learn throughout the next 6 months.
- How do we submit the exercise?

Prepare yourself from today

Q. Have you prepared your resume yet?

Q. Do you have a LinkedIn account, and are you an active LinkedIn user?

Never stop problem solving

How Modern Web Apps Work

A familiar moment

You open Twitter. Tweets appear in under a second.

What actually happens between your tap and that feed filling the screen?

Two Computers, Talking

Client

Your phone or laptop

Runs the Twitter app / browser

Only displays what it's given

the internet



Server

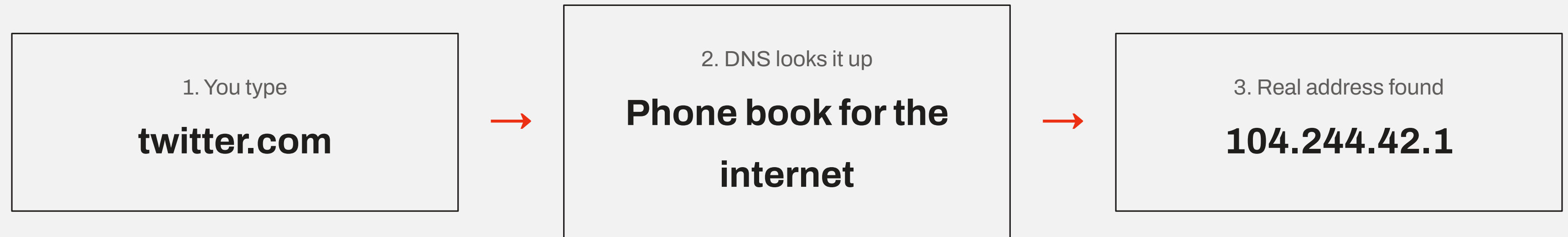
A computer at Twitter, always on

Holds the real data and logic

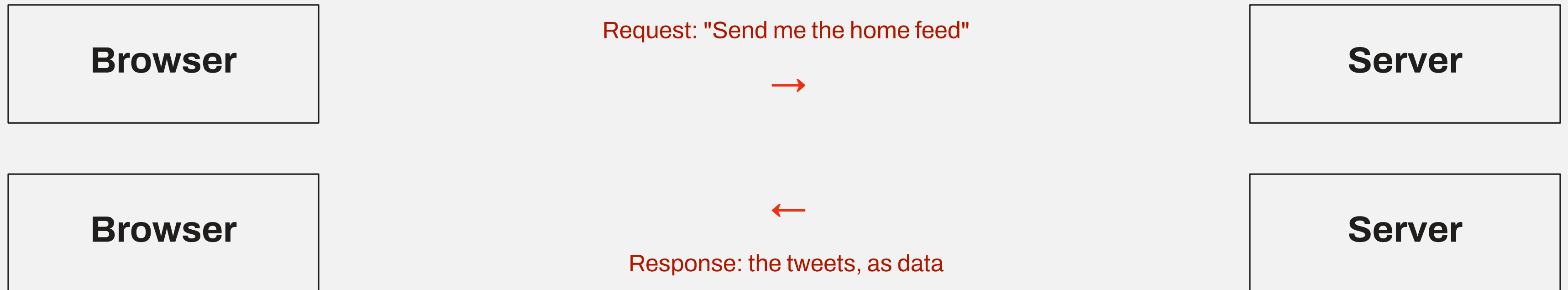
Answers many clients at once

From Name to Address: DNS

A name like twitter.com isn't an address — it has to be looked up.



The Request / Response Cycle



Frontend vs Backend

Frontend

Runs on the client, in the browser

Draws the screen: layout, buttons, feed

Built with HTML, CSS, JavaScript

Backend

Runs on the server

Decides what data each request gets

Talks to the database, applies the rules

Where the Tweets Live: the Database

Server

Runs the backend code



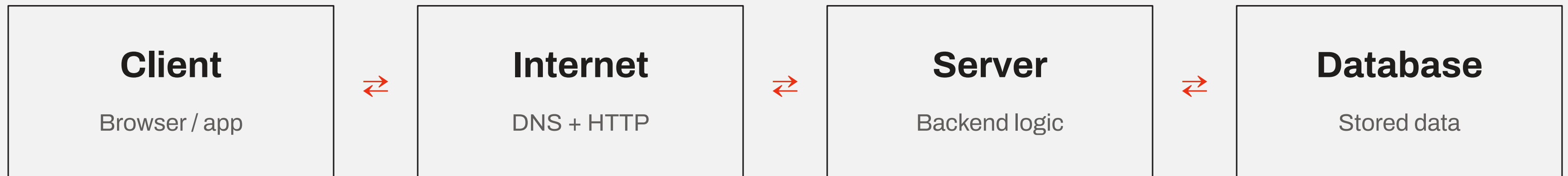
Database

Every tweet, user, and like — stored as structured records

Tracing It: Opening Your Feed

- 1** You type twitter.com and hit enter
- 2** DNS finds the server's real address
- 3** Browser sends a request over the internet
- 4** Server receives it, backend code runs
- 5** Backend asks the database for your tweets
- 6** Data comes back; the browser renders your feed

The Full Picture



Every web app moves data through these same four pieces.

**Get an overview of the tools,
technologies, and libraries we'll use.**

1. Programming Languages & Frameworks

1. C#
2. HTML
3. CSS
4. Javascript
5. SQL
6. .NET
7. next.js

2. Object-Oriented Programming

1. Classes & Objects
2. Encapsulation
3. Inheritance
4. Polymorphism
5. Interfaces

3. Database

1. DBMS Fundamentals
2. Relational Databases
3. PostgreSQL
4. SQL Queries
5. Relationships
6. Indexing
7. Transactions

4. .NET & Libraries

1. .NET Web API
2. Entity Framework Core
3. FluentValidation
4. SignalR
5. Serilog
6. Hangfire
7. MassTransit

5. Authentication & Security

1. Authentication
2. Authorization
3. JWT Authentication
4. Role-Based Authorization

6. Messaging & Caching (advanced)

1. RabbitMQ

2. Redis

7. Software Architecture & Design

1. SOLID Principles
2. CQRS
3. Clean Architecture
4. Repository Pattern
5. Design Best Practices

8. DevOps & Deployment

1. Git
2. GitHub
3. Docker
4. CI/CD

9. Hosting & Cloud

1. Render
2. Vercel
3. Neon Database
4. Production Configuration

10. AI for Developers

1. Prompt Engineering
2. Using AI for Development
3. Increasing Developer Productivity with AI

**Analyze the requirements behind
popular Twitter features.**

Twitter As A Product

Treating Twitter as a client requirement — and deciding what we will and won't build.

What is Twitter, Really?

NOT THIS

A product you scroll on your phone/browser?

THIS

A system a client has hired us to build

Core Requirements

1. As a user, I can create account & login and manage my profile.
2. As a user, I can post, edit and delete tweet.
3. As a user, I can like and retweet posts.
4. As a user, I can follow other users and see their tweets in my feeds.
5. As a user, I can see real-time notifications.

Core Requirements

6. As a user, I can search for user and tweets.
7. As a user, I can view other users profiles.
8. As a user, I can send and receive real time message with my friends.
9. As a user, I can bookmark tweets to read later.

**See how each Twitter feature maps
to the concepts we'll learn
throughout the next 6 months.**

1. Create Account, Login & Profile

- Database
- Backend server
- Password hashing
- Authentication
- JWT
- Entity Framework Core
- FluentValidation
- Frontend

2. Post, edit and delete tweet

- Tweet table
- Relationship between user and tweet
- Authorization so only the owner can edit or delete their tweets.
- Logging
- RabbitMQ

3. Like and retweet

- Prevent duplicate like

4. Follow other users and see their tweets

- Query optimization
- Indexing
- Pagination

5. See real-time notification

- SignalR

6. Search for users

- Sorting
- Filtering
- Full-text search

How can users access our application over the internet?

- Neon
- Vercel
- Render
- CI/CD

How do we submit the exercise?

Home works

1. A short definition of each concept from Agenda 3.

Thank you all!